

# รายงานโครงการ ระบบจุกะซ์ร้าน KPP (Yum Jukebox)

ระบบจุกะซ์สำหรับร้านกาแฟที่ลูกค้าขอเพลงผ่าน QR Code

ทดสอบใช้งานได้ที่ <https://yumpakjure.onrender.com/>  
ที่เก็บซอร์สโค้ด <https://github.com/tlemtv098-dotcom/YumparkjureQRsong>

ชื่อ-นามสกุล นักศึกษา: นายสุวัฒน์ จันไพจิตร

รหัสประจำตัวผู้เรียน: 671320101

รายงานโครงการ

เพื่อการส่งมอบและประเมินผล

กำหนดส่ง 15 ตุลาคม 2569 (15 October 2026)

# บทที่ 1 บทนำ

## 1.1 ความเป็นมาและปัญหาที่ต้องการแก้ไข

ร้านกาแฟและร้านอาหารที่มีโซนนั่งรอ มักใช้ระบบจุกะซ์หรือเครื่องเล่นเพลงอัตโนมัติเพื่อให้ลูกค้า เลือกฟังเพลงได้เอง แต่เครื่องจุกะซ์ที่มีขายทั่วไปมักมีข้อจำกัดสำคัญ 3 ข้อ ได้แก่ รายการเพลงถูก ตรึงไว้

ผู้ใช้โทรศัพท์ที่เป็นเครื่องของร้านเองหรือหน้าเครื่องตั้ง และไม่มีประวัติว่าเพลงใดถูกขอบ่อย

ร้านจึงเสียโอกาสที่จะรู้ความนิยมของลูกค้า

ปัญหาที่ต้องการแก้ไขจึงเป็นการหาวิธีที่จะทำให้ลูกค้าในร้านส่งคำขอเพลงได้เองจากโทรศัพท์ของตน โดยไม่ต้องเดินไปที่เคาน์เตอร์ เพื่อให้คิวเพลงถูกจัดลำดับอย่างเป็นระบบ และให้เจ้าของร้านเห็นภาพรวมของความนิยมนั้นผ่านกราฟ โครงการนี้จึงนำเสนอระบบจุกะซ์ที่ทำงานบนเว็บ ลูกค้าสแกนคิวอาร์โค้ดที่ติด บนโต๊ะ แล้วเปิดหน้าเว็บไซต์เพื่อค้นหาและขอเพลง เพลงที่ขอเข้ามาจะถูกเรียงลงคิวบนหน้าจอเครื่องเล่นในร้าน และเจ้าของร้านเปิดดูสถิติจากแดชบอร์ดได้

## 1.2 วัตถุประสงค์ของโครงการ

1. ให้ลูกค้าสมัครสมาชิก เข้าสู่ระบบ และออกจากระบบได้
2. กำหนดบทบาทผู้ใช้อย่างน้อย 2 บทบาท และควบคุมสิทธิการเข้าถึงแต่ละส่วนจริง
3. สร้างกระบวนการขอเพลงที่ลูกค้าทำได้ ตั้งแต่ค้นหาเพลงจนเพลงเข้าคิว
4. จัดการข้อมูลเพลง ผู้ใช้ แนวเพลง และเท็กด้วย Django Model และฐานข้อมูล
5. ให้เจ้าของร้านเห็นภาพรวมความนิยมของเพลงผ่านกราฟบนแดชบอร์ด
6. ให้บริการผ่านอินเทอร์เน็ต เพื่อให้ลูกค้าใช้งานได้จากอุปกรณ์ของตนเอง

## 1.3 ขอบเขตของระบบ

ระบบทำงานเป็นเว็บแอปแบบ Server-Rendered ของ Django ประกอบด้วยหน้าเว็บสำหรับผู้ใช้ทุกบทบาท และ REST API จำนวน 26 เส้นทางสำหรับให้ส่วนติดต่อกับระบบโดยไม่ต้องโหลดหน้าใหม่ทั้งหน้า

ขอบเขตของงานครอบคลุมตั้งแต่การสมัครสมาชิก การค้นหาเพลงจาก YouTube การเพิ่มเพลงเข้าคิว

การเล่นเพลงบนหน้าจอ การจัดการเพลย์ลิสต์ การจัดการผู้ใช้ แนวเพลง และเท็กของผู้ดูแลระบบ

รวมถึงแดชบอร์ดสรุปสถิติและการส่งออกข้อมูลรูปแบบ CSV

รายการที่อยู่นอกขอบเขตคือระบบชำระเงิน การส่งคำขอเพลงผ่านอีเมล และการจัดการสิทธิ์แบบตามฟังก์ชัน สำหรับผู้ใช้ทุกคน เพราะโครงการนี้เน้นที่การควบคุมสิทธิ์ตามบทบาทซึ่งทำได้ครบและตรวจสอบได้ชัดเจนกว่า

## 1.4 ความสำคัญและประโยชน์ที่ได้รับ

- ลดเวลาที่ลูกค้าต้องเดินไปที่เคาน์เตอร์เพื่อขอเพลง
- ทำให้คิวเพลงยุติธรรมขึ้น เพราะทุกคำขอถูกบันทึกเป็นลำดับเวลา
- เจ้าของร้านทราบความนิยมของเพลงจากกราฟ โดยไม่ต้องเปิดหูฟังเพลงทั้งหมด

- เก็บประวัติการขอเพลงไว้วิเคราะห์ต่อได้ และเป็นฐานสำหรับการจัดเพลย์ลิสต์ประจำวัน

## บทที่ 2 การวิเคราะห์และออกแบบระบบ

### 2.1 บทบาทผู้ใช้งานและสิทธิ์การเข้าถึง

ระบบกำหนดผู้ใช้ 3 บทบาท เก็บไว้ในฟิลด์ role ของโมเดล Profile ได้แก่ admin, staff และ customer การควบคุมสิทธิ์ทำงานด้วยมอดูลศาสตร์ role\_required ในไฟล์ music/views\_auth.py ซึ่งจะตรวจสอบว่าผู้ใช้เข้าสู่ระบบแล้วหรือยัง และบทบาทอยู่ในรายการที่อนุญาตหรือไม่ หากไม่ผ่านจะแจ้งข้อความและพากลับไปหน้าเครื่องเล่น ทันที ไม่ใช่แค่ซ่อนปุ่มในหน้าเว็บ

| บทบาท    | สิทธิ์ที่ได้รับ                                                                                                                  | สิทธิ์ที่ถูกปฏิเสธ                                                                 |
|----------|----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| admin    | ดูแลแปดอร์ จัดการผู้ใช้ (เพิ่ม แก้ไข ปิดใช้งาน ลบ)<br>จัดการแนวเพลงและแท็ก จัดการคิวเพลง<br>เปิดแดชบอร์ดและกราฟ ส่งออกข้อมูล CSV | ไม่สามารถลบบัญชีของตนเองที่เป็นผู้ดูแลระบบคนสุดท้ายได้                             |
| staff    | จัดการคิวเพลง ย้ายลำดับเพลง ล้างคิว จัดการเพลย์ลิสต์<br>เปิดหน้าเมนูของฝ่ายบริหาร                                                | ไม่สามารถสร้าง แก้ไข หรือลบผู้ใช้<br>และไม่สามารถจัดการแนวเพลงกับแท็กได้           |
| customer | สมัครและเข้าสู่ระบบ ขอเพลง ดูเพลงที่ตนสั่ง ลบเพลงที่ตนสั่ง<br>สร้างและจัดการเพลย์ลิสต์ของตน<br>แก้ไขโปรไฟล์และรหัสผ่าน           | ไม่สามารถเข้าถึงหน้าจัดการผู้ใช้ แนวเพลง แท็ก<br>แดชบอร์ด และคิวเพลงรวมของทั้งร้าน |

การตรวจสอบสิทธิ์ที่ฝั่งเซิร์ฟเวอร์เป็นสิ่งสำคัญ เพราะการซ่อนปุ่มด้วย HTML หรือ JavaScript ป้องกันได้แค่ระดับหน้าตา ผู้ใช้ที่เข้าถึง URL โดยตรงยังต้องผ่านการตรวจสอบจากมอดูลศาสตร์นี้ ชุดทดสอบ tests\_nav\_customer.py มีกรณีตรวจสอบว่าบทบาท customer เปิด /management/users/ ไม่ได้จริง

### 2.2 กรณีการใช้งาน (Use Case)

| ผู้ใช้  | กรณีการใช้งาน                  | ผลลัพธ์ที่ระบบต้องทำ                                                                      |
|---------|--------------------------------|-------------------------------------------------------------------------------------------|
| ลูกค้า  | สมัครสมาชิกใหม่                | สร้าง User และ Profile บทบาท customer โดยอัตโนมัติ พร้อมเข้าสู่ระบบทันที                  |
| ลูกค้า  | ค้นหาเพลงที่อยากฟัง            | เรียก YouTube Data API v3 แล้วกรองเฉพาะวิดีโอที่ฟังได้และเป็นเพลง พร้อมส่งผลเป็น JSON     |
| ลูกค้า  | กดขอเพลง                       | สร้างแถวใน SongQueue พร้อมชื่อผู้ขอ และคืนสถานะสำเร็จเป็น JSON                            |
| ลูกค้า  | เล่นเพลงถัดไปในหน้าเครื่องเล่น | เล่นเพลงถัดไปผ่าน YouTube IFrame API แล้วเรียก /api/played/ เพื่อทำเครื่องหมายว่าเล่นแล้ว |
| ลูกค้า  | สร้างเพลย์ลิสต์และเพิ่มเพลง    | สร้าง Playlist และเขียน PlaylistSong พร้อมกำหนด position ตามลำดับที่เพิ่ม                 |
| พนักงาน | จัดการคิวเพลงหน้าเครื่องเล่น   | ย้ายลำดับเพลง ลบเพลง หรือล้างคิวทั้งหมดผ่านมอดูลศาสตร์ role_required                      |

| ผู้ใช้  | กรณีการใช้งาน             | ผลลัพธ์ที่ระบบต้องทำ                                                                    |
|---------|---------------------------|-----------------------------------------------------------------------------------------|
| ผู้ดูแล | เปิดแดชบอร์ด              | คำนวณจำนวนผู้ใช้ ผู้ใช้ที่ใช้งานอยู่ จำนวนแนวเพลง และจำนวนเพลง พร้อมส่งข้อมูลกราฟ 4 ชุด |
| ผู้ดูแล | เพิ่มหรือลบแนวเพลงและแท็ก | ทำ CRUD ผ่านฟอร์มที่ตรวจสอบความถูกต้องของ slug และความยาวชื่อก่อนบันทึก                 |
| ผู้ดูแล | ส่งออกข้อมูล              | ส่งออกเพลย์ลิสต์และคิวเพลงเป็นไฟล์ CSV ที่ดาวน์โหลดได้ทันที                             |

## 2.3 สถาปัตยกรรมของระบบโดยรวม

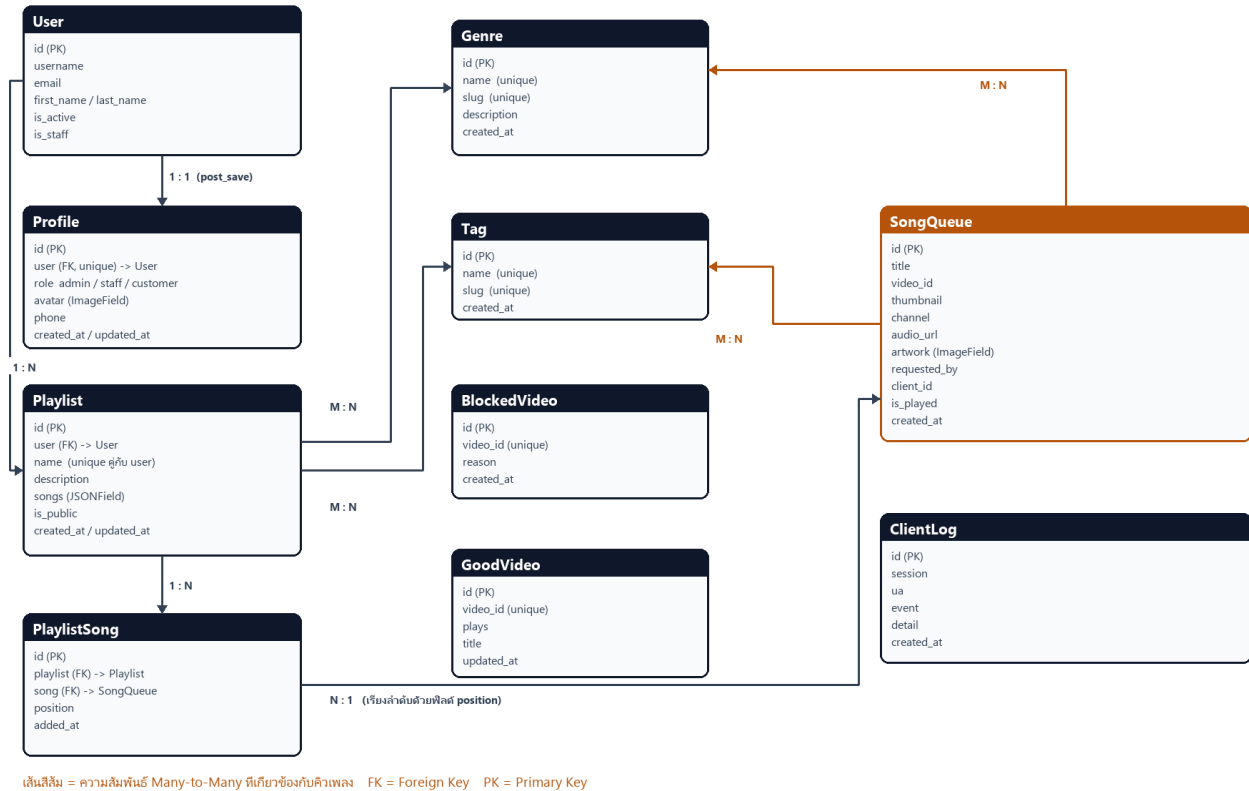
ระบบแบ่งเป็น 4 ชั้น ได้แก่ ชั้นนำเสนอ (Template) ที่สร้างหน้า HTML ด้วย Tailwind CSS ชั้นตรรกะ (View) ที่ประมวลผลคำขอและส่งข้อมูลกลับ ชั้นโมเดล (Model) ที่กำหนดโครงสร้างข้อมูลและชุดฟอร์ม และชั้นฐานข้อมูลที่เป็น SQLite เมื่อมีการตั้งค่า DATABASE\_URL ระบบจะสลับไปใช้ฐานข้อมูลภายนอกผ่านแพ็คเกจ dj-database-url โดยอัตโนมัติ

- ผู้ใช้เปิดเว็บไซต์ ระบบส่งหน้าเข้าสู่ระบบและหน้าสมัครสมาชิก
- ผู้ใช้ค้นหาเพลง ระบบเรียก YouTube Data API v3 แล้วกรองผลลัพธ์ฝั่งเซิร์ฟเวอร์
- ผู้ใช้กดขอเพลง ระบบบันทึกลงโมเดล SongQueue และหน้าเครื่องเล่นดึงคิวมาแสดงผล
- หน้าเครื่องเล่นเรียกเพลงถัดไปมาเล่นผ่าน YouTube IFrame API แล้วทำเครื่องหมายว่าเล่นแล้ว
- เจ้าของร้านเปิดแดชบอร์ด ระบบคำนวณสถิติแล้วส่งเป็น JSON ไปยัง Chart.js วาดกราฟ

## 2.4 แบบจำลองข้อมูล

ระบบมีโมเดล 9 โมเดล และมีความสัมพันธ์ครบทั้งสามแบบที่โจทย์กำหนด ดังภาพ

### แผนภาพความสัมพันธ์ของข้อมูล (Entity Relationship Diagram)



ภาพที่ 2-1 แผนภาพความสัมพันธ์ของข้อมูล (Entity Relationship Diagram)

## 2.5 ความสัมพันธ์ระหว่างโมเดลที่ใช้งานจริง

- One-to-One: Profile.user ซึ่งไปยัง User ด้วย ForeignKey ที่มีค่า unique แลของ Profile ถูกสร้างอัตโนมัติโดย signal ที่รับ post\_save ของ User เพื่อไม่ให้ผู้ใช้ทุกคนต้องไปกรอกข้อมูลโปรไฟล์เอง
- One-to-Many: Playlist.user ซึ่งไปยัง User หนึ่งผู้ใช้มีเพลย์ลิสต์ได้หลายชุด และ SongQueue เก็บ requested\_by ไว้ว่าเพลงนั้นถูกสั่งโดยใคร
- Many-to-Many: Playlist.genres, Playlist.tags, SongQueue.genres และ SongQueue.tags ซึ่งไปยัง Genre และ Tag ทำให้เพลงหนึ่งเพลงจัดหมวดหมู่ได้หลายหมวด และหมวดหมู่หนึ่งอาจรวมหลายเพลง
- Many-to-Many ผ่านตารางกลาง: ความสัมพันธ์ระหว่าง Playlist กับ SongQueue ใช้โมเดล PlaylistSong เป็นตารางกลาง ซึ่งเก็บฟิลด์ position เพื่อจำลำดับเพลงในเพลย์ลิสต์ วิธีนี้ต่างจากการใช้ ManyToManyField ตรง ๆ ตรงที่เก็บข้อมูลเพิ่มเติมของความสัมพันธ์ได้ เช่น ลำดับและเวลาที่เพิ่ม

ข้อสังเกตที่ควรรู้: Playlist ยังมีฟิลด์ songs แบบ JSONField เก็บรายการเป็น JSON ไว้ด้วย เพื่อรองรับโค้ดเดิมที่อ่านค่านี้ ส่วนความสัมพันธ์จริงที่ใช้งานอยู่คือตาราง PlaylistSong ฟังก์ชัน add\_song เขียนทั้งสองส่วนให้ตรงกันเสมอ

## 2.6 ขั้นตอนการทำงานของการทำงานขอเพลง

## ขั้นตอนการทำงานตั้งแต่ค้นหาจนเพลงเข้าคิว



ภาพที่ 2-2 ขั้นตอนการทำงานตั้งแต่ค้นหาจนเพลงเข้าคิว

## บทที่ 3 การพัฒนาระบบ

### 3.1 สภาพแวดล้อมการพัฒนา

| รายการ                | ค่าที่ใช้                                                                                                                 |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------|
| ภาษาหลักของเฟรมเวิร์ก | Python 3.12 (Dockerfile ใช้ python:3.12-slim)                                                                             |
| เฟรมเวิร์กเว็บ        | Django 6.0.6                                                                                                              |
| เว็บเซิร์ฟเวอร์       | Gunicorn 21.2.0 (2 workers)                                                                                               |
| แพ็คเกจประกอบ         | yt-dlp, qrcode, pillow, whitenoise, dj-database-url, psycopg                                                              |
| ฐานข้อมูล             | SQLite (db.sqlite3) และสลับไปใช้ฐานข้อมูลภายนอกเมื่อตั้ง DATABASE_URL                                                     |
| แคช                   | LocMemCache                                                                                                               |
| ไฟล์นิ่ง              | Tailwind CSS, Chart.js, JavaScript ของเทมเพลต                                                                             |
| แพลตฟอร์ม             | Render (แผนฟรี) เข้าถึงได้ที่ <a href="https://yumpakjure.onrender.com/">https://yumpakjure.onrender.com/</a>             |
| ที่เก็บซอร์สโค้ด      | <a href="https://github.com/tlemtv098-dotcom/YumparkjureQRsong">https://github.com/tlemtv098-dotcom/YumparkjureQRsong</a> |
| ระบบควบคุมเวอร์ชัน    | Git (242 คอมมิต)                                                                                                          |

โครงสร้างโปรเจกต์แยกตามหน้าที่ของ Django ไฟล์ `yum_jukebox/settings.py` เก็บค่าตั้งและรวม URL แอป music ประกอบด้วยโมเดล ฟอรัม วิว เทมเพลต และคำสั่งจัดการฐานข้อมูล รวมถึงชุดทดสอบที่แยกตามหัวข้อ จำนวน 9 ไฟล์

### 3.2 โมเดลและโครงสร้างข้อมูล

โมเดลทั้ง 9 โมเดลอยู่ในไฟล์ `music/models.py` รายละเอียดของแต่ละโมเดลดังตาราง

| โมเดล    | ฟิลด์หลัก                                                                | ความสัมพันธ์                                                          |
|----------|--------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Profile  | user, role, avatar, phone, created_at, updated_at                        | One-to-One -> User (สร้างอัตโนมัติด้วยสัญญาณ <code>post_save</code> ) |
| Genre    | name, slug, description, created_at                                      | ปลายทางของ Many-to-Many จาก Playlist และ SongQueue                    |
| Tag      | name, slug, created_at                                                   | ปลายทางของ Many-to-Many จาก Playlist และ SongQueue                    |
| Playlist | user, name, description, songs (JSON), is_public, created_at, updated_at | One-to-Many (user -> User), Many-to-Many -> Genre/Tag                 |

| โมเดล        | ฟิลด์หลัก                                                                                               | ความสัมพันธ์                                                      |
|--------------|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| PlaylistSong | playlist, song, position, added_at                                                                      | ตารางเชื่อม Playlist <-> SongQueue แบบ Many-to-Many ผ่านตารางกลาง |
| SongQueue    | title, video_id, thumbnail, channel, audio_url, artwork, requested_by, client_id, is_played, created_at | One-to-Many (ผู้ถูกบันทึกคำขอ), Many-to-Many -> Genre/Tag         |
| BlockedVideo | video_id, reason, created_at                                                                            | ตัวกรอง video_id ที่ฟังไม่ได้                                     |
| GoodVideo    | video_id, plays, title, updated_at                                                                      | บันทึกวิดีโอที่เล่นได้จริง                                        |
| ClientLog    | session, ua, event, detail, created_at                                                                  | บันทึกเหตุการณ์ฝั่งผู้ใช้                                         |

การเปลี่ยนแปลงโครงสร้างข้อมูลทั้งหมดถูกบันทึกเป็น migration จำนวน 10 ไฟล์ ตั้งแต่ 0001\_initial จนถึง 0010\_playlistsong เมื่อนำขึ้นใช้งานระบบจะรันคำสั่ง migrate ก่อนเริ่ม Unicorn ตามที่กำหนดไว้ใน Procfile

### 3.3 ระบบสมาชิกและการยืนยันตัวตน

การสมัครสมาชิกใช้ RegisterForm ที่สืบทอดจาก UserCreationForm ของ Django เพิ่มฟิลด์อีเมล ชื่อ นามสกุล เบอร์โทร และคำยืนยันรหัสผ่าน เมื่อบันทึกสำเร็จ สัญญาณ post\_save จะสร้างแถว Profile ให้อัตโนมัติ และวิวจะกำหนดบทบาทเป็น customer จากนั้นจึงเรียก login เพื่อเข้าสู่ระบบทันที

การเข้าสู่ระบบใช้ LoginView ของ Django ร่วมกับ ThaiLoginForm ที่ปรับข้อความแจ้งเตือนเป็นภาษาไทย เส้นทาง /accounts/logout/ ใช้ LogoutView และพากลับไปหน้าเข้าสู่ระบบเมื่อออกจากระบบ

### 3.4 การควบคุมสิทธิ์ด้วย role\_required

```
def role_required(allowed_roles):
    def decorator(view_func):
        @login_required
        def _wrapped_view(request, *args, **kwargs):
            if not hasattr(request.user, 'profile'):
                return redirect('player')
            if request.user.profile.role not in allowed_roles:
                return redirect('player')
            return view_func(request, *args, **kwargs)
        return _wrapped_view
    return decorator
```

มอดูลศาสตร์นี้ถูกใช้กับบริหารจัดการผู้ใช้ แนวเพลง แท็ก แดชบอร์ด และการส่งออกข้อมูล ทำให้ผู้ที่ไม่มีสิทธิ์เข้าถึงไม่ได้แม้จะพิมพ์ URL โดยตรง และไม่ได้พึ่งการซ่อนเมนูเพียงอย่างเดียว

### 3.5 การทำ CRUD

ระบบทำ CRUD ได้ 5 กลุ่ม ได้แก่ ผู้ใช้ แนวเพลง แท็ก เพลย์ลิสต์ และคิวเพลง ทุกกลุ่มใช้ฟอร์มของ Django ร่วมกับมีกิตติศาสตร์ role\_required ทุกการแก้ไขมีการตรวจสอบความถูกต้องก่อนบันทึก และแสดงข้อความแจ้งเตือนผ่านระบบ messages ของ Django เมื่อบันทึกสำเร็จหรือไม่สำเร็จ

- ผู้ใช้: เพิ่มผู้ใช้ แก้ไขข้อมูลและบทบาท ปิดใช้งานชั่วคราว และลบผู้ใช้
- แนวเพลง: เพิ่ม แก้ไข ลบ พร้อมช่องค้นหาและการแบ่งหน้า
- แท็ก: เพิ่ม แก้ไข ลบ พร้อมช่องค้นหาและการแบ่งหน้า
- เพลย์ลิสต์: สร้าง เพิ่มเพลง ลบเพลง และสลับลำดับเพลงในเพลย์ลิสต์
- คิวเพลง: เพิ่มหน้าเดิมหรือต่อท้าย ย้ายลำดับ ทำเครื่องหมายว่าเล่นแล้ว และล้างคิว

### 3.6 การค้นหาเพลงผ่าน YouTube Data API v3

ฟังก์ชัน youtube\_api\_search เรียก <https://www.googleapis.com/youtube/v3/search> โดยส่งพารามิเตอร์ part=snippet, type=video, regionCode=TH, videoCategoryId=10 (หมวดเพลง), videoEmbeddable=true และ videoSyndicated=true การกำหนด videoEmbeddable ทำให้ได้เฉพาะวิดีโอที่ฝังในหน้าเว็บได้จริง ซึ่งเป็นการกรองปัญหาที่พบบ่อยที่สุดของระบบยูเคซ์

ระบบรองรับคีย์หลายตัวโดยอ่านจากตัวแปรสภาพแวดล้อม YOUTUBE\_API\_KEYS ค้นด้วยเครื่องหมายจุลภาค เมื่อคีย์ตัวหนึ่งโควตาหมดหรือถูกจำกัดอัตราการเรียก ระบบจะลองคีย์ถัดไปอัตโนมัติ และกำหนดเวลารอของคำขอไว้ที่ 8 วินาที เมื่อไม่มีคีย์หรือเรียกไม่สำเร็จ ระบบจะใช้รายการเพลงสำรองที่เตรียมไว้ในโค้ดแทน เพื่อให้หน้าค้นหายังใช้งานได้ต่อ

หลังได้ผลลัพธ์ ระบบกรองซ้ำอีกชั้นฝั่งเซิร์ฟเวอร์ ได้แก่ ตรวจสอบรูปแบบ video\_id ด้วยนิพจน์ปกติ ข้ามวิดีโอที่อยู่ในตาราง BlockedVideo ข้ามชื่อที่ดูเป็นเนื้อหาสร้างด้วย AI และข้ามคลิปที่ไม่ใช่เพลง การกรองขั้นนี้ทำให้ผลลัพธ์ที่แสดงแก่ลูกค้าสะอาดกว่าการกรองฝั่งหน้าเว็บ

### 3.7 การอัปโหลดไฟล์และรูปภาพ

ระบบรองรับการอัปโหลดไฟล์ 2 จุด ได้แก่ รูปประจำตัวของผู้ใช้ เก็บในฟิลด์ Profile.avatar และภาพปกของเพลง เก็บในฟิลด์ SongQueue.artwork โดยฟังก์ชัน clean\_avatar ตรวจสอบ 3 เงื่อนไขคือ ขนาดไฟล์ไม่เกิน 2 MB ชนิดไฟล์ต้องเป็น JPEG, PNG หรือ WebP และความกว้างสูงไม่เกิน 1000 พิกเซล การตรวจสอบขนาดภาพใช้ Pillow ซึ่งเป็นแพ็คเกจที่ติดตั้งไว้ใน requirements.txt

### 3.8 ระบบแบ่งหน้า (Pagination)

หน้าจัดการผู้ใช้ แนวเพลง และแท็ก ใช้ Paginator ของ Django หน้าละ 20 รายการ รับค่าหน้าจากพารามิเตอร์ ?page= และใช้เมธอด get\_page เพื่อให้เลขหน้าที่เกินจำนวนหน้าจริงถูกจัดการอย่างปลอดภัย หน้าจัดการผู้ใช้อังยังรองรับการค้นหาจากชื่อผู้ใช้และอีเมล พร้อมตัวกรองตามบทบาทและสถานะการใช้งาน

### 3.9 การตรวจสอบความถูกต้องของฟอร์ม

การตรวจสอบทำทั้งจากตัวตรวจสอบมาตรฐานของ Django และจากเมธอด clean เพิ่มเติม ดังนี้

- อีเมลต้องไม่ซ้ำกับบัญชีที่มีอยู่ (clean\_email)

- เบอร์โทรต้องเป็นตัวเลขเท่านั้น โดยตัดเครื่องหมายขีดและช่องว่างออกก่อนตรวจ (clean\_phone)
- ความยาวชื่อแนเพลงไม่เกิน 50 ตัวอักษร และ slug ต้องไม่ซ้ำ
- ไฟล์รูปภาพต้องไม่เกิน 2 MB ต้องเป็น JPEG, PNG หรือ WebP และความกว้างสูงไม่เกิน 1000 พิกเซล (clean\_avatar)
- รหัสผ่านต้องผ่านตัวตรวจสอบความซับซ้อนของ Django และตรงกับคำยืนยัน

### 3.10 REST API

ระบบเปิด REST API จำนวน 26 เส้นทางภายใต้ /api/ ทุกเส้นทางตอบกลับเป็น JSON ยกเว้นสองเส้นทางที่คืนไฟล์ คือ /api/audio/ ที่คืนสตรีมเสียง และ /qr.png ที่คืนภาพรหัส QR การทำงานร่วมกันระหว่างหน้าเว็บกับ API เกิดจากการเรียกแบบไม่ต้องโหลดหน้าใหม่ทั้งหน้า (AJAX) ผ่าน fetch ในไฟล์ JavaScript ของเทมเพลต

| กลุ่ม           | ตัวอย่าง endpoint                                                   | หน้าที่                                             |
|-----------------|---------------------------------------------------------------------|-----------------------------------------------------|
| ค้นหา           | /api/search/ , /api/suggest/ , /api/hits/                           | ค้นหาเพลง เติมนำเสนอ และดึงเพลงยอดนิยม              |
| จัดการคิว       | /api/add/ , /api/add-front/ , /api/queue/ , /api/queue/move/        | เพิ่มเพลงต่อคิวหรือต่อหน้า ดูปิด และสลับลำดับ       |
| สถานะเพลง       | /api/played// , /api/clear/ , /api/duration/                        | ทำเครื่องหมายว่าเล่นแล้ว ล้างคิว และอ่านความยาวเพลง |
| เพลงของฉัน      | /api/my-songs/ , /api/my-songs//delete/                             | ดูและลบเพลงที่ตนเป็นผู้ขอ                           |
| เพลย์ลิสต์      | /api/playlists/ , /api/playlists//load/ , /api/playlists//add-song/ | สร้าง โหลด และเติมเพลงลงเพลย์ลิสต์                  |
| ดูและระบบ       | /api/block// , /api/unblock// , /api/stats/                         | บล็อกหรือปลดบล็อกวิดีโอ และดูสถิติรวม               |
| บันทึกเหตุการณ์ | /api/clientlog/ , /api/clientlog/recent/                            | บันทึกและอ่านเหตุการณ์ฝั่งผู้ใช้                    |
| สื่อและอื่น ๆ   | /api/dashboard/stats/ , /api/audio/ , /api/ai/recommend/            | สถิติแดชบอร์ด สตรีมเสียง และคำแนะนำเพลง             |

### 3.11 แดชบอร์ดและกราฟ

หน้า /dashboard/ แสดงการ์ดสรุป 4 ใบ ได้แก่ จำนวนผู้ใช้ทั้งหมด ผู้ใช้ที่ใช้งานอยู่ จำนวนแนเพลง และจำนวนเพลงในคิว จากนั้นเป็นกราฟ 4 กราฟที่สร้างด้วย Chart.js ได้แก่

กราฟแท่งแนวนอนแสดงจำนวนเพลงแยกตามแนเพลง กราฟเส้นแสดงจำนวนคำขอเพลงย้อนหลัง 7 วัน

กราฟแท่งแนวนอนแสดงเพลงยอดนิยม 10 อันดับแรก และกราฟวงแหวนแสดงสัดส่วนสถานะคิวเพลง

ข้อมูลสำหรับกราฟถูกคำนวณและแปลงเป็น JSON ในวิวฝั่งเซิร์ฟเวอร์ ไม่ใช่คำนวณใน JavaScript ของหน้าเว็บ ทำให้ตรวจสอบความถูกต้องได้จากชุดทดสอบ

### 3.12 การส่งออกข้อมูล

หน้าแดชบอร์ดมีปุ่มส่งออกข้อมูล 2 ปุ่มที่ใช้งานได้จริง ได้แก่ /export/playlists/csv/ ที่ส่งออกรายการเพลย์ลิสต์ และ /export/queue/csv/ ที่ส่งออกคิวเพลง ทั้งสองเส้นทางอยู่ภายใต้มัลติพาสเวิร์ด role\_required และคืนไฟล์ชนิด text/csv ให้ดาวน์โหลดทันที ส่วนปุ่มส่งออกเป็น PDF ที่ปรากฏอยู่ในหน้าแดชบอร์ดยังไม่ได้เชื่อมต่อ กับโค้ดจัดการ จึงยังใช้งานไม่ได้ ซึ่งระบุไว้ในข้อจำกัดของบทที่ 6

### 3.13 ความปลอดภัยที่เพิ่มเติม

- ตรวจสอบสิทธิ์ซ้ำที่ฝั่งเซิร์ฟเวอร์ด้วย role\_required ไม่พึ่งเพียงการซ่อนเมนู
- ตรวจสอบขนาดและชนิดไฟล์ที่อัปโหลดก่อนบันทึกลงดิสก์
- ป้องกันการอ่านไฟล์นอกโฟลเดอร์ media ด้วยการตรวจสอบด้วย os.path.commonpath ใน yum\_jukebox/urls.py
- ป้องกัน CSRF ด้วย CsrfViewMiddleware และมีชุดทดสอบ XSS แยก 10 เคส
- เตือนเมื่อ SECRET\_KEY ยังเป็นค่าสำรองในโค้ด และเตือนเมื่อ DEBUG เปิดอยู่ในสภาพแวดล้อมการใช้งานจริง
- ใช้ WhiteNoise เสริมไฟล์นิ่งและ Gunicorn เป็นเว็บเซิร์ฟเวอร์ในการใช้งานจริง

### 3.14 การนำขึ้นใช้งานจริง

ระบบถูกนำขึ้นใช้งานบนแพลตฟอร์ม Render ที่ <https://yumpakjare.onrender.com/>

ซึ่งเข้าถึงได้จากอินเทอร์เน็ตทั่วโลก ไฟล์ Procfile กำหนดให้รันคำสั่ง migrate และ ensure\_admin ก่อนเริ่ม Gunicorn ทุกครั้งที่เริ่มระบบ นอกจากนี้ยังมี Dockerfile และไฟล์ railway.toml

เตรียมไว้สำหรับการติดตั้งแบบคอนเทนเนอร์บนแพลตฟอร์มอื่น ทั้งนี้เว็บไซต์ที่ใช้งานจริงทำงานบน Render โดยตรง

## บทที่ 4 ผลการพัฒนาและภาพหน้าจอ

### 4.1 ความครบถ้วนตามข้อกำหนดของงาน

| ข้อกำหนดของงาน                             | สิ่งที่ระบบทำได้                                                                  | ตำแหน่งในโค้ด                                                                           |
|--------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| สมัครสมาชิก / เข้าสู่ระบบ / ออกจากระบบ     | ครบทั้งสามส่วน พร้อมฟอร์มภาษาไทยและข้อความแจ้งเตือน                               | music/views_auth.py<br>, music/views.py<br>(ThaiLoginForm)                              |
| อย่างน้อย 2 บทบาท<br>พร้อมควบคุมสิทธิ์จริง | 3 บทบาท (admin, staff, customer) คุมด้วย role_required                            | music/views_auth.py<br>บรรทัด role_required                                             |
| CRUD ด้วย Django Model<br>และฐานข้อมูล     | CRUD ครบ ทั้งผู้ใช้ แนวเพลง แท็ก เพลย์ลิสต์ และคิวเพลง                            | music/views_auth.py<br>, music/views.py                                                 |
| ระบบค้นหา                                  | ค้นหาเพลงผ่าน YouTube Data API v3<br>พร้อมตัวกรองฝั่งเซิร์ฟเวอร์                  | music/views.py<br>ฟังก์ชัน<br>youtube_api_search<br>และ search_youtube                  |
| อัปโหลดไฟล์ / รูปภาพ                       | อัปโหลดรูปประจำตัว (avatar) และภาพปกเพลง (artwork)<br>พร้อมตรวจสอบขนาดและชนิดไฟล์ | music/forms.py<br>ฟังก์ชัน clean_avatar                                                 |
| แบ่งหน้า (Pagination)                      | Paginator หน้าละ 20 รายการ ทั้งผู้ใช้ แนวเพลง และแท็ก                             | music/views_auth.py<br>ฟังก์ชัน<br>user_list_view,<br>genre_list_view,<br>tag_list_view |
| ความสัมพันธ์ระหว่างโมเดล                   | มีครบ One-to-One, One-to-Many และ Many-to-Many                                    | music/models.py                                                                         |
| ตรวจสอบความถูกต้องของฟอร์ม                 | ตรวจสอบอีเมลซ้ำ เบอร์โทรผิดรูป ขนาดไฟล์ ชนิดไฟล์ ความยาว slug                     | music/forms.py                                                                          |
| ส่วนติดต่อผู้ใช้ที่ใช้งานได้จริง           | หน้าเข้าสู่ระบบ สมัครสมาชิก แถบเมนู และหน้า<br>ติดต่อ/แจ้งปัญหาในหน้าเครื่องเล่น  | music/templates/ba<br>se.html, music/temp<br>lates/music/player.h<br>tml                |
| ให้บริการบนอินเทอร์เน็ต                    | ใช้งานจริงที่ Render ผ่าน Gunicorn และ WhiteNoise                                 | Procfile, Dockerfile,<br>yum_jukebox/settin<br>gs.py                                    |

### 4.2 ภาพหน้าจอของระบบ

ภาพทั้งหมดต่อไปนี้เป็นที่กจากระบบที่รันจริงด้วยข้อมูลตัวอย่างที่สร้างจากคำสั่ง `ensure_admin --with-demo` โดยเข้าสู่ระบบในส่วนที่ต้องการสิทธิ์ด้วยบัญชี `admin` เพื่อให้เห็นหน้าจอจริงทั้งหมด



## เข้าสู่ระบบเจ้าของร้าน

ล็อกอินเพื่อจัดการระบบคิวเพลง

**ชื่อผู้ใช้**

**รหัสผ่าน**  
 [แสดง](#)

**เข้าสู่ระบบ**

ภาพที่ 4-1 หน้าเข้าสู่ระบบ พร้อมข้อความแจ้งเตือนภาษาไทย



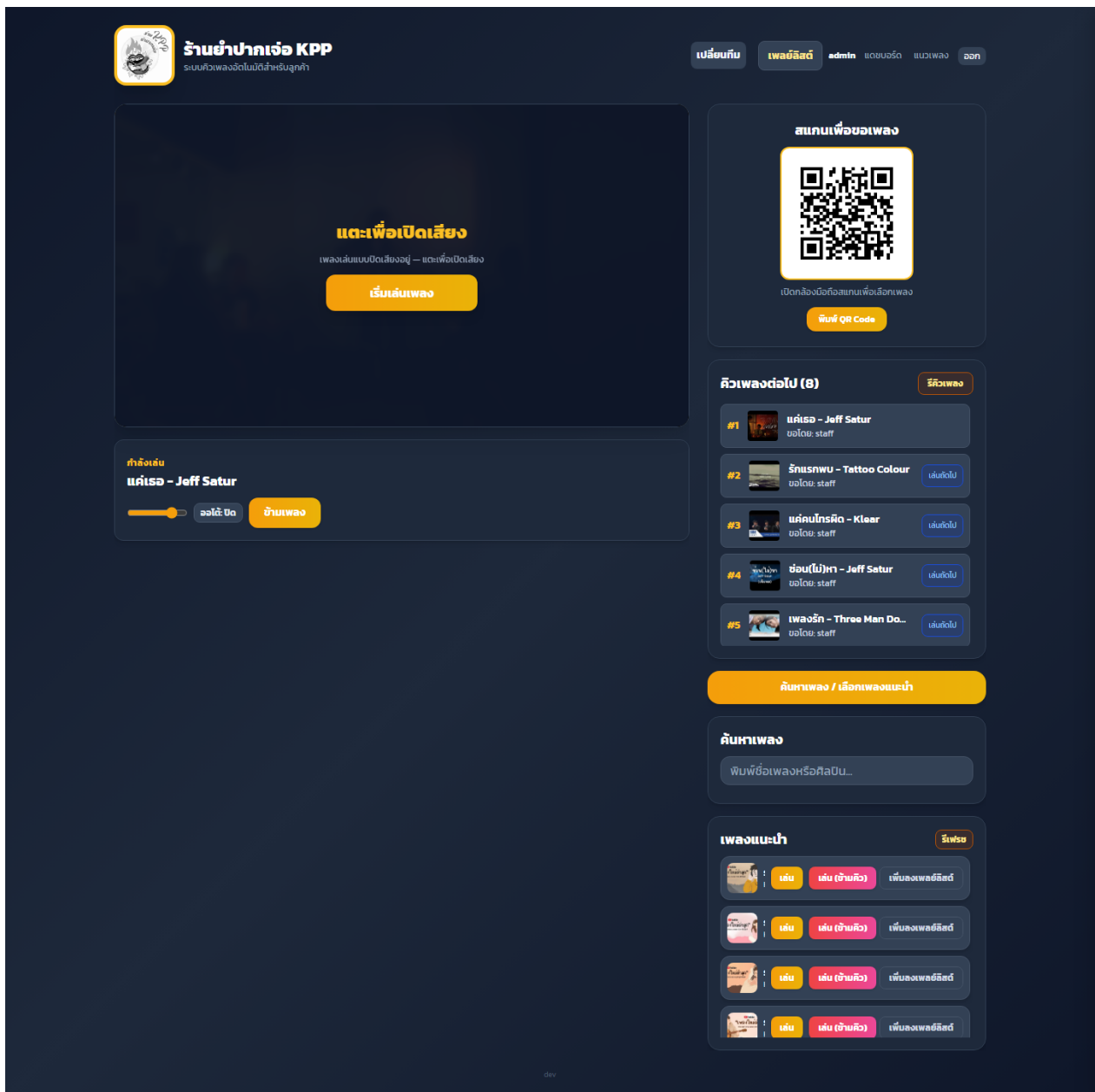
## สมัครสมาชิก

สร้างบัญชีเพื่อใช้งานระบบคิวเพลง

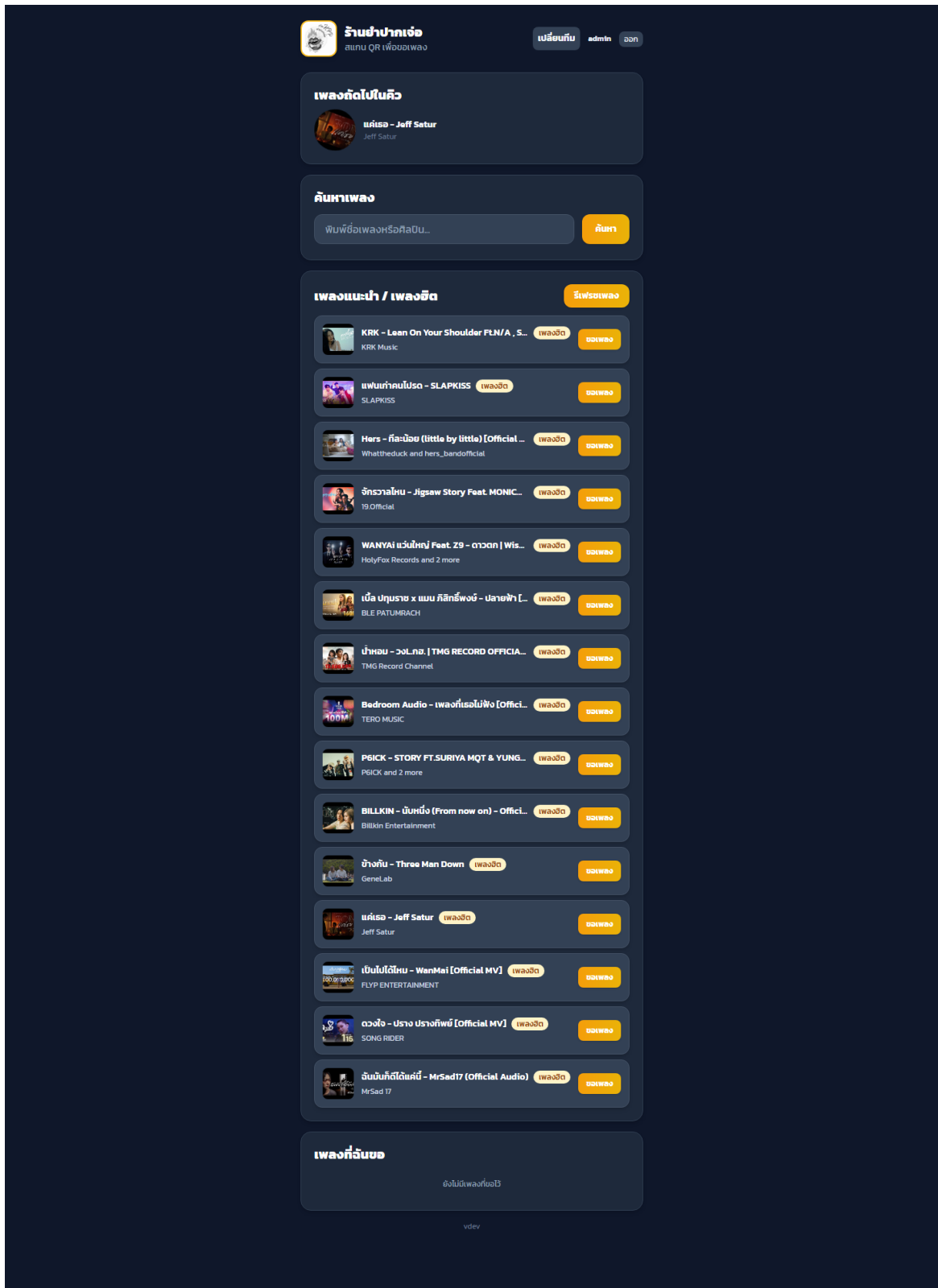
|                                            |                      |
|--------------------------------------------|----------------------|
| ชื่อ                                       | นามสกุล              |
| <input type="text"/>                       | <input type="text"/> |
| ชื่อผู้ใช้                                 |                      |
| <input type="text"/>                       |                      |
| อีเมล                                      |                      |
| <input type="text"/>                       |                      |
| เบอร์โทร (ไม่บังคับ)                       |                      |
| <input type="text"/>                       |                      |
| รหัสผ่าน                                   |                      |
| <input type="password"/>                   |                      |
| ยืนยันรหัสผ่าน                             |                      |
| <input type="password"/>                   |                      |
| <input type="button" value="สมัครสมาชิก"/> |                      |

มีบัญชีแล้ว? [เข้าสู่ระบบ](#)

ภาพที่ 4-2 หน้าสมัครสมาชิก มีช่องอีเมล ชื่อ นามสกุล เบอร์โทร และคำยืนยันรหัสผ่าน



ภาพที่ 4-3 หน้าเครื่องเล่นจูเกะซึขณะเข้าสู่ระบบในฐานะ admin เมื่อกดเล่นแล้วระบบจะเล่นเพลงถัดไป



ภาพที่ 4-4 หน้าขอเพลง ผลลัพธ์จาก YouTube พร้อมรูปปก ซองใส่เท็ก และปุ่มขอเพลง

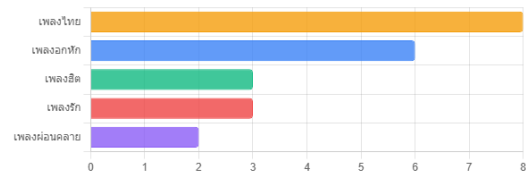
## แดชบอร์ด

รีเฟรช

ผู้ใช้ทั้งหมด  
3ผู้ใช้ที่ใช้งานอยู่  
3เพลย์ลิสต์ทั้งหมด  
4คิวรอเล่น  
8

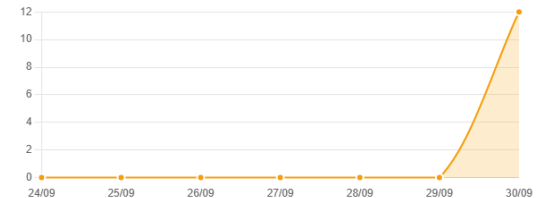
## แนวเพลงยอดนิยม

อันดับล่าสุด



## คิวเพลง 7 วันที่ผ่านมา

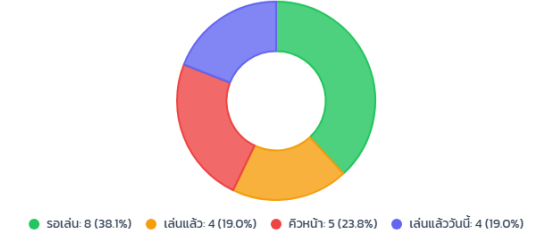
7 วัน



## เพลงยอดนิยม (Top 10)



## สถานะคิวเพลง



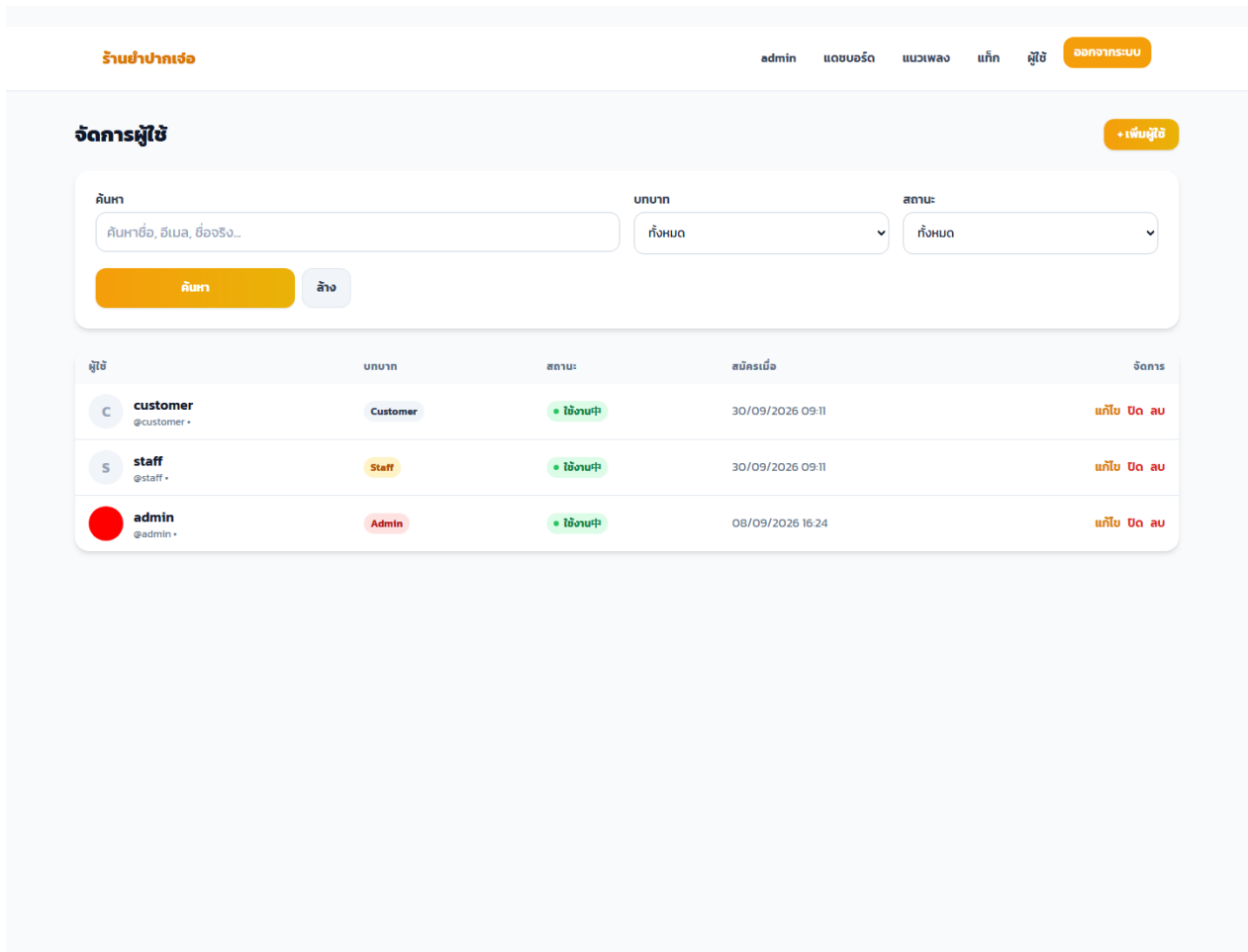
## ส่งออกข้อมูล (Bonus)

ส่งออกเพลย์ลิสต์ (CSV)

ส่งออกคิวเพลง (CSV)

ส่งออกแดชบอร์ด (PDF)

ภาพที่ 4-5 แดชบอร์ดของผู้ดูแลระบบ ประกอบด้วยการ์ดสรุป 4 ใบ กราฟ 4 กราฟ และปุ่มส่งออกข้อมูล



ภาพที่ 4-6 หน้าจัดการผู้ใช้ มีช่องค้นหา ตัวกรองตามบทบาทและสถานะ และปุ่มแก้ไข ปิดใช้งาน ลบ

## จัดการแนวเพลง

+ เพิ่มแนวเพลง

ค้นหาแนวเพลง...

ค้นหา

ล้าง

| แนวเพลง                                       | คำอธิบาย                            | เพลย์ลิสต์ | จัดการ                                   |
|-----------------------------------------------|-------------------------------------|------------|------------------------------------------|
| <b>เพลงพ่อนคลาย</b><br>slug: chill            | เพลงเบา ๆ สำหรับฟังตอนทานข้าว       | 0          | <a href="#">แก้ไข</a> <a href="#">ลบ</a> |
| <b>เพลงรัก</b><br>slug: love-songs            | เพลงรักและเพลงความรัก               | 0          | <a href="#">แก้ไข</a> <a href="#">ลบ</a> |
| <b>เพลงออกหัก</b><br>slug: lump-in-the-throat | เพลงเศร้า ๆ ที่ทำให้ใจพังก          | 0          | <a href="#">แก้ไข</a> <a href="#">ลบ</a> |
| <b>เพลงฮิต</b><br>slug: hits                  | เพลงที่คนในร้านชอบฟังที่สุด         | 0          | <a href="#">แก้ไข</a> <a href="#">ลบ</a> |
| <b>เพลงแดนซ์</b><br>slug: dance               | เพลงจังหวะแรงสำหรับงานเต้น          | 0          | <a href="#">แก้ไข</a> <a href="#">ลบ</a> |
| <b>เพลงไทย</b><br>slug: thai-pop              | เพลงไทยยอดนิยมที่ลูกค้าในร้านชอบฟัง | 0          | <a href="#">แก้ไข</a> <a href="#">ลบ</a> |

ภาพที่ 4-7 หน้าจัดการแนวเพลง (CRUD) พร้อมช่องค้นหาและปุ่มเพิ่มแนวเพลง

## จัดการแท็ก

+ เพิ่มแท็ก

ค้นหาแท็ก...

ค้นหา

ล้าง

| แท็ก                                   | เพลงฮิสต์ | จัดการ                                   |
|----------------------------------------|-----------|------------------------------------------|
| <b>ข้างกัน</b><br>slug: beside-you     | 0         | <a href="#">แก้ไข</a> <a href="#">ลบ</a> |
| <b>คิดถึง</b><br>slug: thinking-of-you | 0         | <a href="#">แก้ไข</a> <a href="#">ลบ</a> |
| <b>รักแรกพบ</b><br>slug: first-love    | 0         | <a href="#">แก้ไข</a> <a href="#">ลบ</a> |
| <b>สนุก</b><br>slug: fun               | 0         | <a href="#">แก้ไข</a> <a href="#">ลบ</a> |
| <b>อกหัก</b><br>slug: heartbreak       | 0         | <a href="#">แก้ไข</a> <a href="#">ลบ</a> |
| <b>เพลงรัก</b><br>slug: romance        | 0         | <a href="#">แก้ไข</a> <a href="#">ลบ</a> |

ภาพที่ 4-8 หน้าจัดการแท็ก (CRUD) ซึ่งใช้รูปแบบเดียวกับหน้าแนวเพลง

## บทที่ 5 การทดสอบ

### 5.1 การทดสอบอัตโนมัติ

การทดสอบอัตโนมัติเขียนด้วย unittest ของมาตรฐาน Django จำนวน 314 เคส แยกเป็น 17 โมดูลตามหัวข้อ เพื่อให้เห็นชัดว่าแต่ละส่วนถูกตรวจสอบอะไรบ้าง คำสั่งที่ใช้รันคือ

```
venv\Scripts\python.exe manage.py test
```

ผลการรันครั้งสุดท้าย

```
Found 314 test(s).
Ran 314 tests in 279.9s

OK
```

| โมดูลการทดสอบ              | หน้าที่ที่ตรวจ                                            | จำนวนเคส |
|----------------------------|-----------------------------------------------------------|----------|
| tests.py                   | เส้นทางหลัก การขอเพลง คิวเพลง การค้นหา เพลย์ลิสต์ API     | 156      |
| tests_demo_accounts.py     | คำสั่ง ensure_admin และบัญชีทดสอบ 3 บทบาท                 | 28       |
| tests_production_config.py | ค่าตั้งค่าสำหรับ production (SECRET_KEY, DEBUG, security) | 15       |
| tests_nav_customer.py      | เมนูและสิทธิ์ของบทบาท customer                            | 11       |
| tests_xss.py               | การป้องกัน XSS ในช่องค้นหาและข้อมูลที่แสดงผล              | 10       |
| tests_dashboard.py         | แดชบอร์ดและการคำนวณค่าสถิติ                               | 8        |
| tests_media.py             | การอัปโหลดไฟล์และการเสิร์ฟไฟล์ media                      | 7        |
| tests_nav.py               | เมนูของบทบาท staff และ admin                              | 6        |
| tests_profile_signal.py    | สัญญาณ post_save สร้าง Profile อัตโนมัติ                  | 6        |
| รวม                        | ผลการรัน: Ran 314 tests — OK                              | 314      |

### 5.2 รายละเอียดของการทดสอบในแต่ละด้าน

- tests.py จำนวน 156 เคส ครอบคลุมเส้นทางหลักทั้งหมด ทั้งหน้าเว็บ การขอเพลง การจัดการคิว การค้นหา และเพลย์ลิสต์ รวมถึงกรณีผิดปกติ เช่น ส่ง JSON ไม่ถูกต้อง หรือไม่มีสิทธิ์เข้าถึง
- tests\_demo\_accounts.py จำนวน 28 เคส ตรวจสอบคำสั่ง ensure\_admin สร้างบัญชีทั้งสามบทบาท และข้อมูลตัวอย่างได้ถูกต้อง และรันซ้ำแล้วไม่ทำให้ข้อมูลเพี้ยน
- tests\_production\_config.py จำนวน 15 เคส ตรวจสอบการตั้งค่าที่จำเป็นต่อการใช้งานจริง เช่น การเตือนเมื่อ SECRET\_KEY เป็นค่าสำรอง และการปิด DEBUG ในสภาพแวดล้อมการใช้งานจริง

- tests\_nav\_customer.py และ tests\_nav.py จำนวน 17 เคส ตรวจสอบเมนูและการเข้าถึงหน้าต่าง ๆ เป็นไปตามบทบาทจริง
- tests\_xss.py จำนวน 10 เคส ตรวจสอบการป้องกันสคริปต์ที่ส่งมากับคำค้นและข้อมูลที่แสดงผล
- tests\_media.py จำนวน 7 เคส ตรวจสอบการอัปโหลดไฟล์และการเสิร์ฟไฟล์ในโฟลเดอร์ media
- tests\_dashboard.py จำนวน 8 เคส ตรวจสอบการคำนวณสถิติและการควบคุมสิทธิ์ของแต่ละบอร์ด
- tests\_profile\_signal.py จำนวน 6 เคส ตรวจสอบว่าสัญญาณ post\_save สร้าง Profile ให้ผู้ใช้ใหม่เสมอ

### 5.3 ผลการทดสอบด้วยมือ

นอกจากการทดสอบอัตโนมัติแล้ว ยังทำการทดสอบด้วยมือตามตารางต่อไปนี้ เพื่อยืนยันพฤติกรรมที่ผู้ใช้เห็นจริง โดยเฉพาะกรณีที่การทดสอบอัตโนมัติตรวจไม่ได้ เช่น การเล่นเพลงบนอุปกรณ์จริงและการแสดงผลบนหน้าจอ

| ลำดับ | สิ่งที่ต้องทำ                                 | ผลที่คาดหวัง                                                                    |
|-------|-----------------------------------------------|---------------------------------------------------------------------------------|
| 1     | เปิดหน้าเข้าสู่ระบบแล้วกรอก admin / 12345678  | เข้าสู่ระบบได้ เมนูด้านบนเพิ่มเมนูจัดการ                                        |
| 2     | กดออกจากระบบ แล้วกรอกรหัสผิด                  | ขึ้นข้อความขอผู้ใช้หรือรหัสผ่านไม่ถูกต้อง                                       |
| 3     | สมัครสมาชิกใหม่ด้วยอีเมลที่ยังไม่เคยใช้       | สมัครได้ และเข้าสู่ระบบอัตโนมัติ                                                |
| 4     | สมัครใหม่อีกครั้งด้วยอีเมลเดิม                | ขึ้นข้อความ อีเมลนี้ถูกใช้งานแล้ว                                               |
| 5     | กรอกเบอร์โทรศัพท์เป็นตัวอักษร                 | ขึ้นข้อความ เบอร์โทรต้องเป็นตัวเลขเท่านั้น                                      |
| 6     | หน้า /request/ พิมพ์คำค้น แล้วกดค้นหา         | แสดงผลลัพท์เป็นรายการพร้อมรูปปกและช่องใส่แท็ก                                   |
| 7     | กด ขอเพลง ที่ผลลัพธ์ใดผลลัพธ์หนึ่ง            | เพลงถูกเพิ่มเข้าคิว และถูกลบออกจากรายการผลลัพธ์                                 |
| 8     | กด ขอเพลง ซ้ำกับเพลงเดิม                      | ไม่เพิ่มซ้ำ ระบบตรวจจับเพลงที่อยู่ในคิวแล้ว                                     |
| 9     | เปิดหน้า / แล้วกดเล่นเพลงถัดไป                | เพลงถัดไปขึ้นเป็นเพลงที่กำลังเล่น และสถานะเปลี่ยนเป็นเล่นแล้ว                   |
| 10    | กดแท็กหนึ่งในรายการเพลงที่เล่นได้ล่าสุด       | เพลงนั้นถูกโหลดเข้าคิวและเริ่มเล่น                                              |
| 11    | กด ล้างคิวเพลง                                | คิวว่าง และปุ่มยืนยันถูกซ่อน                                                    |
| 12    | หน้า /dashboard/ ตรวจสอบกราฟทั้งสี่           | กราฟแท่งแนวนอน กราฟเส้น กราฟแท่ง Top 10 และกราฟวงแหวน แสดงข้อมูลตรงกับฐานข้อมูล |
| 13    | หน้า /management/users/ ค้นหาด้วยคำว่า admin  | รายการเหลือเฉพาะผู้ใช้ที่ตรงกับคำค้น                                            |
| 14    | หน้า /management/users/ เปลี่ยนบทบาทของผู้ใช้ | บทบาทในตารางเปลี่ยนตามที่เลือก                                                  |
| 15    | หน้า /management/genres/ เพิ่มแนวเพลงใหม่     | แนวเพลงใหม่ปรากฏในตารางพร้อม slug                                               |
| 16    | ลบแนวเพลงที่ยังไม่ถูกใช้งาน                   | แนวเพลงถูกลบออกจากตาราง                                                         |
| 17    | หน้า /accounts/profile/ อัปโหลดรูปเกิน 2 MB   | ขึ้นข้อความ ไฟล์รูปภาพต้องไม่เกิน 2 MB                                          |
| 18    | อัปโหลดไฟล์ .txt เป็นรูปประจำตัว              | ขึ้นข้อความ รองรับเฉพาะไฟล์ JPEG, PNG, WebP เท่านั้น                            |

| ลำดับ | สิ่งที่ต้องทำ                                           | ผลที่คาดหวัง                                     |
|-------|---------------------------------------------------------|--------------------------------------------------|
| 19    | เข้าสู่ระบบด้วย customer แล้วเปิด<br>/management/users/ | ถูกพากลับไปหน้าเครื่องเล่น พร้อมข้อความแจ้งเตือน |
| 20    | เปิด /qr.png                                            | ได้ภาพรหัส QR ที่สแกนแล้วเปิดหน้าเว็บไซต์ได้     |

## บทที่ 6 สรุปผล

### 6.1 สิ่งทีบรรลุตามวัตถุประสงค์

โครงการนี้พัฒนาระบบยูเกะซึสำหรับร้านกาแฟที่ลูกค้าใช้สแกนคิวอาร์โค้ดเพื่อขอเพลงได้ด้วยตนเอง โดยไม่ต้องเดินไปที่เคาน์เตอร์ ระบบทำงานได้ครบตามข้อกำหนดของงานทุกข้อ คือ การสมัครสมาชิก เข้าสู่ระบบและออกจากระบบ การควบคุมสิทธิ์ตาม 3 บทบาท CRUD ด้วย Django Model และฐานข้อมูล ระบบค้นหา การอัปโหลดไฟล์ การแบ่งหน้า ความสัมพันธ์ระหว่างโมเดล การตรวจสอบความถูกต้องของฟอร์ม ส่วนติดต่อผู้ใช้ที่อ่านง่ายและใช้งานได้จริง และการให้บริการผ่านอินเทอร์เน็ต

นอกจากข้อกำหนดหลัก ระบบยังมีองค์ประกอบตามเกณฑ์พิจารณาพิเศษ ได้แก่

| เกณฑ์พิจารณาพิเศษ       | สถานะ    | รายละเอียดที่มีในระบบ                                                                                                 |
|-------------------------|----------|-----------------------------------------------------------------------------------------------------------------------|
| REST API                | มี       | เส้นทางได้ /api/ จำนวน 26 เส้นทาง ครอบคลุมการค้นหา คิวเพลง เพลย์ลิสต์ และสถิติ                                        |
| การเชื่อมต่อ API ภายนอก | มี       | YouTube Data API v3 สำหรับค้นหา และ YouTube oEmbed สำหรับข้อมูลวิดีโอ                                                 |
| แดชบอร์ดหรือกราฟ        | มี       | แดชบอร์ดพร้อมกราฟ 4 กราฟ วาดด้วย Chart.js                                                                             |
| การค้นหาขั้นสูง         | มี       | กรองตามประเทศและหมวดเพลง ตรวจสอบสถานะการฟังได้ กรองคีย์เวิร์ด และมีระบบเติมคำแนะนำอัตโนมัติ                           |
| การทดสอบอัตโนมัติ       | มี       | ชุดทดสอบ 314 เคส 17 โมดูล ผ่านทั้งหมด                                                                                 |
| การแจ้งเตือนทางอีเมล    | ยังไม่มี | ระบบยังไม่ได้เชื่อมต่อการส่งอีเมล ระบุไว้ในแนวทางพัฒนาต่อ                                                             |
| AJAX                    | มี       | หน้าเว็บเรียก API ด้วย fetch ไม่ต้องโหลดหน้าใหม่ทั้งหน้า                                                              |
| การส่งออกข้อมูล         | บางส่วน  | ส่งออกเพลย์ลิสต์และคิวเพลงเป็น CSV ได้จริง ส่วนปุมส่งออก PDF ยังไม่ได้เชื่อมต่อ                                       |
| Docker                  | มีไฟล์   | Dockerfile และ railway.toml เตรียมไว้ แต่เว็บไซต์จริงทำงานบน Render                                                   |
| CI/CD                   | ยังไม่มี | ยังไม่ได้ตั้งค่า workflow อัตโนมัติใน GitHub Actions                                                                  |
| ความปลอดภัยเพิ่มเติม    | มี       | ตรวจสอบสิทธิ์ฝั่งเซิร์ฟเวอร์ ตรวจสอบไฟล์อัปโหลด ป้องกันการอ่านไฟล์นอกโฟลเดอร์ ป้องกัน XSS และมีชุดทดสอบความปลอดภัยแยก |

ผลการทดสอบอัตโนมัติผ่านทั้งหมด 314 เคส ไม่มีเคสไหนล้มเหลว ระบบถูกนำขึ้นใช้งานจริงและเปิดให้ทดลองใช้งานได้จากอินเทอร์เน็ตที่ <https://yumpakjure.onrender.com/>

### 6.2 ข้อจำกัดที่ทราบของระบบ

ระบบยังมีข้อจำกัดที่ทราบอยู่หลายข้อ ผู้จัดทำรายงานไว้อย่างเปิดเผยทั้งที่สามารถตรวจสอบได้จากโค้ดจริง เพราะการนำเสนอเฉพาะสิ่งที่ดูดีอย่างเดียวจะทำให้ผู้ประเมินตรวจสอบแล้วไม่เชื่อถือระบบทั้งหมด

| ข้อจำกัด                                               | รายละเอียดและผลกระทบ                                                                                                                                                                                                                     | แนวทางแก้ไข                                                                     |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| ฐานข้อมูลบน Render เป็น SQLite บนดิสก์ชั่วคราว         | ทุกครั้งที่เรา deploy ใหม่ ฐานข้อมูลจะถูกลบและสร้างขึ้นใหม่ ข้อมูลที่ถูกค่าของเพลงไว้จะหายไป คำสั่ง ensure_admin และตัวเลือก --with-demo เป็นเพียงการสร้างบัญชีและข้อมูลตัวอย่างขึ้นใหม่ ทุกครั้งที่เราเริ่มระบบ ไม่ใช่การเก็บข้อมูลถาวร | ใช้ฐานข้อมูลภายนอก เช่น PostgreSQL ของ Render หรือ Neon โดยตั้งค่า DATABASE_URL |
| SECRET_KEY เป็นค่าสำรองที่อยู่ในซอร์สโค้ด              | ถ้าไม่ได้ตั้ง SECRET_KEY ในตัวแปรสภาพแวดล้อม ระบบจะยังทำงานต่อ แต่จะเห็น session และโทเคน CSRF ด้วยค่าที่เป็นสาธารณะ โค้ดจะออกค่าเตือนชัดเจนเมื่อพบกรณีนี้                                                                               | ตั้งค่า SECRET_KEY เป็นสตริงสุ่มในตัวแปรสภาพแวดล้อมของ Render                   |
| การเล่นเพลงบน iOS และ Safari มีข้อจำกัดที่ทราบ         | YouTube IFrame API ของบางเครื่อง iOS ตอบกลับเป็น Error 153 ซึ่งเป็นนโยบายฝังผู้ใช้ของ YouTube ไม่ใช่ความผิดของเซิร์ฟเวอร์ ระบบเตรียมโหมด ?noapi=1 ไว้เป็นทางเลือกให้เปลี่ยนไปใช้ iframe ธรรมดา และมีเส้นทางสำรองผ่าน Media Session       | ใช้โหมด ?noapi=1 บนเครื่องที่มีปัญหา รายละเอียดอยู่ใน docs/ios-audio-notes.md   |
| ไฟล์ที่อัปโหลดบน Render ไม่คงอยู่ถาวร                  | ไฟล์ที่อัปโหลดถูกเก็บไว้บนดิสก์ของ container และจะหายไปเมื่อมีการ deploy ใหม่ รูปประจำตัวและภาพปกเพลงที่อัปโหลดไว้จึงไม่อยู่ตลอดไป                                                                                                       | ใช้พื้นที่เก็บไฟล์ภายนอก เช่น S3 หรือ Cloudinary แล้วเก็บแค่ URL ในฐานข้อมูล    |
| ปุ่มส่งออกแดนนอนราย (PDF) บนแดชบอร์ดยังไม่ได้เชื่อมต่อ | ปุ่มปรากฏอยู่ในหน้าแดชบอร์ด แต่ยังไม่มีการจัดการการกด จึงยังส่งออกไฟล์ PDF ไม่ได้ ส่วนการส่งออก CSV สองรายการใช้งานได้จริงที่ /export/playlists/csv/ และ /export/queue/csv/                                                              | เขียน view ที่สร้าง PDF แล้วผูกกับ id ของปุ่มนั้น                               |

## 6.3 แนวทางพัฒนาต่อ

- ย้ายฐานข้อมูลไปใช้ PostgreSQL และย้ายไฟล์ที่อัปโหลดไปยังพื้นที่เก็บไฟล์ภายนอก เพื่อให้ข้อมูลและรูปภาพอยู่คงถาวรข้ามการ deploy
- ตั้งค่า SECRET\_KEY และ DATABASE\_URL เป็นตัวแปรสภาพแวดล้อมในระบบใช้งานจริง
- เชื่อมปุ่มส่งออกแดนนอนรายเป็น PDF ให้ใช้งานได้จริง
- เพิ่มการแจ้งเตือนทางอีเมลเมื่อค่าของเพลงเข้าคิว เพื่อให้ผู้ใช้ทราบลำดับคิวของตน
- เพิ่มการควบคุมโควตาของ YouTube Data API ให้เหมาะกับการใช้งานจริงที่มีผู้ใช้จำนวนมาก

## ภาคผนวก

### ก. ข้อมูลสำหรับเข้าใช้งานระบบ

| รายการ                | ค่า                                                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------|
| เว็บไซต์ที่ใช้งานจริง | <a href="https://yumpakjure.onrender.com/">https://yumpakjure.onrender.com/</a>                                           |
| ที่เก็บซอร์สโค้ด      | <a href="https://github.com/tlemtv098-dotcom/YumparkjureQRsong">https://github.com/tlemtv098-dotcom/YumparkjureQRsong</a> |
| จำนวนคอมมิตใน Git     | 242 คอมมิต                                                                                                                |
| ชุดทดสอบอัตโนมัติ     | 314 เคส ผ่านทั้งหมด แยกเป็น 17 โมดูล                                                                                      |
| จำนวน URL route       | 54 route ใน music/urls.py (ไม่รวม /admin/ ของ Django)                                                                     |
| จำนวน API endpoint    | 26 route อยู่ใต้ /api/                                                                                                    |
| จำนวนโมเดล            | 9 โมเดลใน music/models.py                                                                                                 |
| จำนวน migration       | 10 ไฟล์ ตั้งแต่ 0001_initial ถึง 0010_playlistsong                                                                        |
| บทบาทผู้ใช้           | admin, staff, customer                                                                                                    |
| endpoint รหัส QR      | /qr.png                                                                                                                   |

รหัสคิวอาร์โค้ดสำหรับเปิดหน้าเว็บไซต์จากโทรศัพท์ เข้าถึงได้ที่ <https://yumpakjure.onrender.com/qr.png>



ภาพที่ ก-1 รหัสคิวอาร์โค้ดที่สร้างจากเส้นทาง /qr.png

### ข. บัญชีทดสอบสำหรับผู้ประเมิน

| ชื่อผู้ใช้ | รหัสผ่าน | บทบาท             | สิทธิ์ที่ใช้ได้                                  |
|------------|----------|-------------------|--------------------------------------------------|
| admin      | 12345678 | admin (superuser) | ดูแลปเดออร์, แดชบอร์ด, จัดการผู้ใช้/แนวเพลง/แท็ก |
| staff      | 12345678 | staff             | จัดการคิวเพลงและเพลย์ลิสต์                       |
| customer   | 12345678 | customer          | ขอเพลง สร้างและจัดการเพลย์ลิสต์ของตน             |

ข้อควรทราบเรื่องรหัสผ่าน: คำสั่ง ensure\_admin ที่ไม่มีตัวเลือกเพิ่มเติมจะตั้งรหัสผ่านของบัญชี admin เป็น 11111111 ส่วนเมื่อใช้ตัวเลือก --with-demo รหัสผ่านของบัญชีทั้งสามจะถูกตั้งเป็น 12345678 ตามค่าเริ่มต้นของตัวเลือกนี้ หากต้องการให้ตรงกับตารางข้างต้น ให้รันคำสั่งตามรูปแบบนี้

```
python manage.py ensure_admin
```

```
python manage.py ensure_admin --with-demo
```

## ค. คู่มือการใช้งานย่อ

คู่มือการใช้งานฉบับเต็มจัดทำแยกเป็นอีกเล่มหนึ่งชื่อ คู่มือการใช้งาน.docx สรุปสาระสำคัญไว้ดังนี้

1. สมัครสมาชิก: เปิด <https://yumpakjure.onrender.com/accounts/register/> กรอกอีเมล ชื่อ นามสกุล เบอร์โทรศัพท์ และรหัสผ่าน ระบบจะเข้าสู่ระบบให้อัตโนมัติ
2. เข้าสู่ระบบ: เปิดหน้าเข้าสู่ระบบ แล้วกรอกชื่อผู้ใช้และรหัสผ่าน
3. ขอเพลง: เปิดหน้า ขอเพลง พิมพ์ชื่อเพลงหรือชื่อศิลปิน แล้วกดค้นหา กดปุ่ม ขอเพลง ที่ผลลัพธ์ที่ต้องการ ระบบจะเพิ่มเพลงเข้าคิวของร้าน
4. ใช้หน้าเครื่องเล่น: เปิดหน้าแรกของเว็บไซต์ จะเห็นเครื่องเล่น เพลงที่กำลังเล่น คิวเพลงที่รอ และรายการเพลงที่เล่นล่าสุด กด เพลงถัดไป เพื่อเล่นเพลงถัดไปในคิว
5. เล่นเพลงที่เล่นได้ล่าสุด: กดแท็กในรายการด้านล่าง เพลงนั้นจะถูกโหลดเข้าคิวและเริ่มเล่น
6. ล้างคิว: กดปุ่ม ล้างคิวเพลง ที่มุมขวาบนของกล่องคิว
7. จัดการโปรไฟล์: เปิดหน้า โปรไฟล์ เพื่อแก้ไขชื่อ เบอร์โทรศัพท์ อัปโหลดรูปประจำตัว และเปลี่ยนรหัสผ่าน
8. จัดการโดยผู้ดูแล: ผู้ดูแลเข้า /management/users/ เพื่อเพิ่ม แก้ไข ปิดใช้งาน และลบผู้ใช้ เข้า /management/genres/ และ /management/tags/ เพื่อจัดการแนวเพลงและแท็ก และเปิด /dashboard/ เพื่อดูสถิติและกราฟ

## ง. โครงสร้างไฟล์หลักของโครงการ

| พาร                     | หน้าที่                                                                                           |
|-------------------------|---------------------------------------------------------------------------------------------------|
| manage.py               | จุดเริ่มต้นของคำสั่งจัดการของ Django                                                              |
| yum_jukebox/settings.py | ค่าตั้งระบบ รวมฐานข้อมูล SECRET_KEY และการตรวจค่าความปลอดภัย                                      |
| yum_jukebox/urls.py     | รวม URL ของโปรเจกต์ รวมถึงเส้นทางเสิร์ฟไฟล์ในโพลเดอร์ media                                       |
| music/models.py         | โมเดลทั้ง 9 โมเดลและสัญญาณสร้าง Profile อัตโนมัติ                                                 |
| music/forms.py          | ฟอร์มสมัครสมาชิก โปรไฟล์ ผู้ใช้ เพลย์ลิสต์ แนวเพลง แท็ก และคิวเพลง                                |
| music/views.py          | หน้าเครื่องเล่น การค้นหา การจัดการคิว เพลย์ลิสต์ และ API ฝั่งผู้ใช้                               |
| music/views_auth.py     | สมัครสมาชิก เข้าสู่ระบบ โปรไฟล์ แดชบอร์ด การจัดการผู้ใช้ แนวเพลง แท็ก และมีกฎศาสตร์ role_required |

| พาร                                       | หน้าที่                                            |
|-------------------------------------------|----------------------------------------------------|
| music/urls.py                             | เส้นทางทั้งหมด 54 เส้นทาง                          |
| music/admin.py                            | การลงทะเบียนโมเดลในหน้าผู้ดูแลของ Django           |
| music/management/commands/ensure_admin.py | คำสั่งสร้างหรือซ่อมบัญชีผู้ดูแลและข้อมูลตัวอย่าง   |
| music/templates/                          | เทมเพลตทั้งหมด 15 ไฟล์                             |
| music/static/                             | ไฟล์ JavaScript และไฟล์รูปภาพประจำ                 |
| music/tests*.py                           | ชุดทดสอบอัตโนมัติ 15 ไฟล์                          |
| music/migrations/                         | ไฟล์ migration 11 ไฟล์                             |
| docs/ios-audio-notes.md                   | บันทึกการตรวจสอบปัญหาการเล่นเพลงบน iPad และ Safari |
| Procfile และ Dockerfile                   | คำสั่งเริ่มระบบสำหรับการนำขึ้นใช้งาน               |
| requirements.txt                          | รายการแพ็คเกจที่ระบบต้องใช้                        |